AGLC CRM Phase 1 Implementation Plan

Detailed Roadmap with GitHub Integration

Project: AGLC Law Firm CRM
Phase: Phase 1 - MVP Development
Duration: 8 Weeks (March - May 2026)
Budget: $120,500
Team Size: 3-4 Developers + 1 DevOps + 1 QA

Executive Overview

Phase 1 focuses on delivering a Minimum Viable Product (MVP) with core case management, task tracking, VIP client portal, gamification, wellness features, and the foundational AI agents (Planner and Tracker). The implementation leverages GitHub for version control, CI/CD workflows, and AI agent development with proper documentation and testing.

Week-by-Week Breakdown

Week 1-2: Foundation & Infrastructure Setup

Objectives:

- Set up GitHub repositories and workflows
- Design and implement database schema
- Configure development environment
- Establish CI/CD pipeline

Tasks:

Task	Owner	Duration	Status
Create GitHub organization and repositories	DevOps	1 day	⬜
Set up GitHub Actions for CI/CD	DevOps	2 days	⬜
Design database schema (cases, tasks, users, clients)	Backend Lead	2 days	⬜
Implement database migrations	Backend	1 day	⬜
Set up development environment documentation	DevOps	1 day	⬜
Configure authentication and authorization	Backend	2 days	⬜
Set up logging and monitoring	DevOps	1 day	⬜

GitHub Integration:

- Create main repository: `aglc-crm-main`
- Create AI agents repository: `aglc-crm-ai-agents`
- Create documentation repository: `aglc-crm-docs`
- Set up branch protection rules
- Configure automated testing on pull requests

Deliverables:

- ✅ GitHub repositories initialized
- ✅ Database schema documented
- ✅ Development environment guide
- ✅ CI/CD pipeline operational

Week 3-4: Core Features - Case Management

Objectives:

- Build case management system
- Implement case templates
- Create case lifecycle management

- Develop task management system

Tasks:

Task	Owner	Duration	Status
Design case data model	Backend Lead	1 day	⬜
Implement case CRUD operations	Backend	2 days	⬜
Create case templates system	Backend	2 days	⬜
Build task management system	Backend	2 days	⬜
Implement task status tracking	Backend	1 day	⬜
Create case API endpoints (tRPC)	Backend	2 days	⬜
Build case list and detail UI	Frontend	3 days	⬜
Implement task board UI (Kanban)	Frontend	2 days	⬜
Write unit tests for case operations	QA	2 days	⬜
Write integration tests	QA	2 days	⬜

GitHub Integration:

- Feature branches: `feature/case-management`, `feature/task-system`
- Pull request templates for code review
- Automated testing on each PR
- Code coverage tracking

Deliverables:

- ✅ Case management system operational
- ✅ Task tracking system working
- ✅ API endpoints tested
- ✅ UI components functional

Week 5: AI Agents - Planner & Tracker

Objectives:

- Develop Planner Agent
- Develop Tracker Agent
- Integrate agents with case management
- Set up agent orchestration

Tasks:

Task	Owner	Duration	Status
Design AI agent architecture	AI Lead	1 day	⬜
Implement Planner Agent core logic	AI Dev	2 days	⬜
Implement Tracker Agent core logic	AI Dev	2 days	⬜
Set up agent orchestration engine	AI Dev	1 day	⬜
Integrate agents with case system	Backend	2 days	⬜
Create agent API endpoints	Backend	1 day	⬜
Write agent tests	QA	2 days	⬜
Document agent workflows	Tech Writer	1 day	⬜

GitHub Integration:

- Repository: `aglc-crm-ai-agents`
- Agent-specific branches: `feature/planner-agent`, `feature/tracker-agent`
- Agent documentation in README
- Example prompts and test cases

GitHub Workflow Example:

```yaml
name: AI Agent Tests
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - name: Run agent tests
        run: npm test -- agents/
      - name: Check agent prompts
        run: npm run validate:agents
```

Deliverables:

- ✅ Planner Agent functional
- ✅ Tracker Agent functional
- ✅ Agent orchestration working
- ✅ Agent tests passing

Week 6: VIP Client Portal & Gamification

Objectives:

- Build VIP client portal
- Implement gamification system
- Create reward system
- Set up client communication

Tasks:

Task	Owner	Duration	Status
Design VIP portal UI/UX	Designer	2 days	⬜
Build VIP portal frontend	Frontend	2 days	⬜
Implement client access control	Backend	1 day	⬜
Create gamification data model	Backend	1 day	⬜
Implement Spin the Wheel feature	Frontend	2 days	⬜
Build reward system	Backend	2 days	⬜
Create achievement tracking	Backend	1 day	⬜
Implement leaderboard	Frontend	1 day	⬜
Write gamification tests	QA	1 day	⬜

GitHub Integration:

- Feature branches: `feature/vip-portal`, `feature/gamification`
- Design assets in repository
- Gamification rules documented

Deliverables:

- ✅ VIP portal accessible
- ✅ Gamification system working
- ✅ Rewards system operational
- ✅ Client engagement features live

Week 7: Executive Dashboard & Wellness

Objectives:

- Build executive dashboard
- Implement wellness features
- Create analytics and reporting

- Set up health reminders

Tasks:

Task	Owner	Duration	Status
Design executive dashboard	Designer	1 day	⬜
Build dashboard components	Frontend	2 days	⬜
Implement analytics data collection	Backend	2 days	⬜
Create performance metrics	Backend	1 day	⬜
Build wellness feature UI	Frontend	1 day	⬜
Implement health reminders	Backend	1 day	⬜
Create mood tracking system	Backend	1 day	⬜
Implement prayer/break reminders	Backend	1 day	⬜
Write dashboard tests	QA	1 day	⬜

GitHub Integration:

- Feature branches: `feature/executive-dashboard`, `feature/wellness`
- Dashboard configuration files
- Analytics documentation

Deliverables:

- ✅ Executive dashboard operational
- ✅ Wellness features active
- ✅ Analytics collecting data
- ✅ Reminders working

Week 8: Testing, Documentation & Deployment

Objectives:

- Complete comprehensive testing
- Finalize documentation
- Prepare for deployment
- Conduct user acceptance testing

Tasks:

Task	Owner	Duration	Status
End-to-end testing	QA	2 days	⬜
Performance testing	QA	1 day	⬜
Security testing	Security	1 day	⬜
User acceptance testing	Product	1 day	⬜
Bug fixes and refinements	Dev Team	2 days	⬜
Final documentation	Tech Writer	1 day	⬜
Create deployment guide	DevOps	1 day	⬜
Prepare release notes	Product	1 day	⬜
Deploy to staging	DevOps	1 day	⬜
Final sign-off	Leadership	1 day	⬜

GitHub Integration:

- Create release branch: `release/v1.0.0`
- Tag release in GitHub
- Create GitHub release with notes
- Archive documentation

Deliverables:

- ✅ All tests passing
- ✅ Documentation complete
- ✅ Deployment ready

- ✅ Release notes published

GitHub Repository Structure

Main Repository: `aglc-crm-main`

```
aglc-crm/
├── client/                  # React frontend
│   ├── src/
│   │   ├── pages/          # Page components
│   │   ├── components/     # Reusable components
│   │   ├── hooks/          # Custom hooks
│   │   └── lib/            # Utilities
│   └── public/
├── server/                  # Express backend
│   ├── routers/            # tRPC routers
│   ├── db.ts               # Database queries
│   ├── _core/              # Core infrastructure
│   └── agents/             # AI agent integration
├── drizzle/                # Database schema
│   └── schema.ts
├── shared/                 # Shared types
├── .github/
│   └── workflows/          # GitHub Actions
├── docs/                   # Documentation
└── package.json
```

AI Agents Repository: `aglc-crm-ai-agents`

```
aglc-crm-ai-agents/
├── agents/
│   ├── planner/
│   │   ├── index.ts
│   │   ├── prompts.ts
│   │   ├── types.ts
│   │   └── tests/
│   ├── tracker/
│   │   ├── index.ts
│   │   ├── prompts.ts
│   │   ├── types.ts
│   │   └── tests/
│   ├── crisis-manager/
│   ├── executor/
│   ├── resource/
│   ├── monitor/
│   └── analyzer/
├── orchestration/
│   ├── engine.ts
│   ├── decision-tree.ts
│   └── tests/
├── docs/
│   ├── AGENTS.md
│   ├── ORCHESTRATION.md
│   └── PROMPTS.md
├── examples/
└── package.json
```

Documentation Repository: `aglc-crm-docs`

```
aglc-crm-docs/
├── README.md
├── ARCHITECTURE.md
├── API.md
├── AGENTS.md
├── DATABASE.md
├── DEPLOYMENT.md
├── TROUBLESHOOTING.md
└── images/
```

GitHub Workflows & CI/CD

1. Automated Testing Workflow

```yaml
name: Tests
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - uses: actions/setup-node@v2
      - run: npm install
      - run: npm run test
      - run: npm run test:coverage
      - uses: codecov/codecov-action@v2
```

2. Code Quality Workflow

```yaml
name: Code Quality
on: [push, pull_request]
jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - uses: actions/setup-node@v2
      - run: npm install
      - run: npm run lint
      - run: npm run type-check
```

3. AI Agent Validation Workflow

```yaml
name: AI Agent Validation
on: [push, pull_request]
jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - uses: actions/setup-node@v2
      - run: npm install
      - run: npm run validate:agents
      - run: npm run test:agents
```

4. Security Scanning Workflow

```yaml
name: Security
on: [push, pull_request]
jobs:
  security:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - run: npm audit
      - uses: aquasecurity/trivy-action@master
```

Development Guidelines

Branch Naming Convention

- **Feature:** `feature/feature-name`
- **Bug Fix:** `bugfix/bug-description`
- **Hotfix:** `hotfix/critical-issue`
- **Release:** `release/v1.0.0`
- **Documentation:** `docs/documentation-update`

Commit Message Format

```
<type>(<scope>): <subject>

<body>

<footer>
```

Types: feat, fix, docs, style, refactor, test, chore
Scope: component or module name
Subject: brief description (50 chars max)

Example:

```
feat(case-management): add case template system

Implement template selection and case creation workflow
- Add template selection UI
- Create template data model
- Implement template API endpoints

Closes #123
```

Pull Request Process

1. Create feature branch from `main`
2. Implement feature with tests
3. Push to GitHub
4. Create pull request with description
5. Automated tests must pass
6. Code review required (2 approvals)
7. Merge to `main`
8. Delete feature branch

Testing Strategy

Unit Tests

- **Framework:** Vitest
- **Coverage:** 80%+ target
- **Location:** `*.test.ts` files
- **Command:** `npm run test`

Integration Tests

- **Framework:** Vitest + Supertest
- **Coverage:** API endpoints and workflows
- **Location:** `*.integration.test.ts` files
- **Command:** `npm run test:integration`

End-to-End Tests

- **Framework:** Playwright
- **Coverage:** Critical user flows
- **Location:** `e2e/` directory
- **Command:** `npm run test:e2e`

AI Agent Tests

- **Framework:** Vitest
- **Coverage:** Agent logic and prompts
- **Location:** `agents/*/tests/` directory
- **Command:** `npm run test:agents`

Deployment Strategy

Staging Deployment

- Automatic on merge to `main`
- Full test suite runs
- Manual approval required
- Staging environment URL: `staging.aglc-crm.dev`

Production Deployment

- Manual trigger from release branch
- Requires 2 approvals
- Automated rollback capability
- Production environment URL: `app.aglc-crm.dev`

Deployment Checklist

- ✅ All tests passing
- ✅ Code review approved
- ✅ Documentation updated
- ✅ Database migrations tested
- ✅ Performance benchmarks met
- ✅ Security scan passed
- ✅ Release notes prepared

Risk Mitigation

Risk	Probability	Impact	Mitigation
Scope creep	High	High	Strict feature freeze, weekly reviews
AI agent complexity	Medium	High	Early prototyping, expert consultation
Database performance	Medium	High	Load testing, indexing strategy
Team coordination	Medium	Medium	Daily standups, clear ownership
External API failures	Low	Medium	Fallback mechanisms, monitoring

---

Success Metrics

Technical Metrics

- **Code Coverage:** 80%+
- **Test Pass Rate:** 100%
- **Build Time:** < 5 minutes
- **Deployment Time:** < 10 minutes
- **System Uptime:** 99.9%+

Business Metrics

- **Team Adoption:** 90%+ active users
- **Task Completion:** 95%+ on-time
- **Client Satisfaction:** $^4/_5$ average rating
- **Bug Resolution:** 95%+ within 24 hours
- **Feature Completion:** 100% of MVP features

Team Responsibilities

Backend Lead

- Database design and optimization
- API development and testing
- AI agent integration
- Performance optimization

Frontend Lead

- UI/UX implementation
- Component development
- User experience optimization
- Accessibility compliance

DevOps Engineer

- Infrastructure setup
- CI/CD pipeline management
- Deployment automation
- Monitoring and logging

QA Engineer

- Test strategy and planning
- Automated test development
- Manual testing
- Bug tracking and reporting

AI/ML Developer

- AI agent development

- Prompt engineering
- Agent testing and validation
- Orchestration engine

Tech Writer

- Documentation creation
- API documentation
- User guides
- Troubleshooting guides

Communication Plan

Daily

- **Standup:** 10:00 AM (15 minutes)
- **Format:** What done, what next, blockers
- **Channel:** Slack + Video call

Weekly

- **Sprint Review:** Friday 3:00 PM (1 hour)
- **Sprint Planning:** Monday 10:00 AM (1 hour)
- **Architecture Review:** Wednesday 2:00 PM (1 hour)
- **Channel:** Zoom + Confluence

Bi-weekly

- **Leadership Update:** Thursday 4:00 PM (30 minutes)
- **Channel:** Zoom + Slide deck

GitHub Best Practices

1. **Commit Frequently:** Small, logical commits
2. **Write Clear Messages:** Descriptive commit messages
3. **Review Code:** Thorough pull request reviews
4. **Automate Testing:** Run tests on every commit
5. **Document Changes:** Update docs with features
6. **Tag Releases:** Use semantic versioning
7. **Archive History:** Keep clean git history
8. **Protect Main:** Require reviews before merge

Next Steps

1. ✅ Create GitHub organization and repositories
2. ✅ Set up GitHub Actions workflows
3. ✅ Invite team members to repositories
4. ✅ Configure branch protection rules
5. ✅ Create initial project boards
6. ✅ Schedule kickoff meeting
7. ✅ Begin Week 1 tasks

Document Version: 1.0
Last Updated: March 12, 2026
Status: Ready for Implementation
Approval: Pending Leadership Sign-off

For questions or clarifications, please contact the Project Lead.